

PPD dla opornych

Spis treści

1	Wybrane problemy optymalnego podejmowania decyzji z obszaru badań operacyjnych oraz ich zastosowania w systemach o różnej naturze	2
1.1	Rozdział zasobów w kompleksie operacji niezależnych i zależnych	3
1.2	Szeregowanie zadań niezależnych i niepodzielnych, minimalizacja długości uszeregowania $R C_{\max}$.	4
1.3	Problem przepływowy z kryterium $C_{\max}(F C_{\max})$	5
1.4	Problem komiwojażera	6
1.5	Problem plecakowy 0-1	6
1.6	Problemy rozmieszczenia UFLP, UFLP i p-median	7
1.6.1	Capacitated Facility Location Problem	7
1.6.2	Uncapacitated Facility Location Problem	8
1.6.3	P-median	9
2	Wybrane metody i algorytmy rozwiązywania zagadnień podejmowania decyzji	10
2.1	Wieloetapowe podejmowanie decyzji	11
2.1.1	Sformułowanie problemu wieloetapowego podejmowania decyzji	11
2.1.2	Zasada optymalności Bellmana	11
2.1.3	Programowanie dynamiczne	11
2.2	Wielokryterialne podejmowanie decyzji	12
2.2.1	Wielokryterialne podejmowanie decyzji	12
2.2.2	Front Pareto	12
2.2.3	Skalaryzacja, sprowadzanie do problemów jednokryterialnych	12
2.2.4	Optymalizacja celu głównego	12
2.2.5	Minimalizacja odległości od punktu referencyjnego	12
2.2.6	Metoda AHP	13
2.2.7	Metoda ELECTRE I	14
2.3	Metaheurystyki SA i TS	15
2.3.1	Symulowane wyżażanie	15
2.3.2	Tabu search	16
2.3.3	Pojęcia ogólne	16
2.4	Algorytmy rozwiązania problemów alokacji zasobów w kompleksie operacji niezależnych i zależnych	17
2.4.1	Operacje niezależne	17
2.4.2	Operacje o dowolnej strukturze	18
2.5	Algorytmy szeregowania zadań dla problemu $R C_{\max}$ i dla problemu $F C_{\max}$	19
2.5.1	Algorytm Ibarry	19
2.5.2	Algorytm Johnsona	19
2.5.3	Algorytm NEH (Nawaz Ensore Ham)	19
2.6	Algorytmy najtańszego włączania i 2-optymalny dla TSP	20
2.6.1	Algorytm najtańszego włączania (Cheapest Insertion Algorithm)	20
2.6.2	Algorytm 2-optymalny	20
2.7	Algorytm zachłanny dla 0-1 KP	20
2.8	Algorytm konstrukcyjny dla UFLP	20

Rozdział 1

Wybrane problemy optymalnego podejmowania decyzji z obszaru badań operacyjnych oraz ich zastosowania w systemach o różnej naturze

1.1 Rozdział zasobów w kompleksie operacji niezależnych i zależnych

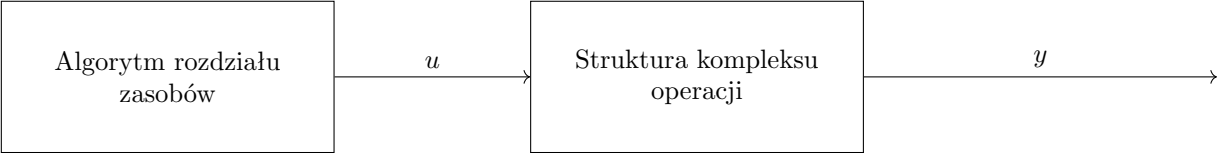
Problem łatwy obliczeniowo dla dowolnej struktury oraz dowolnych modeli operacji. Kompleksy operacji niezależnych i zależnych różnią się od siebie tym, że w operacjach niezależnych operacje wykorzystują zasoby niezależnie. W przypadku kompleksu zadań zależnych zadania mają określoną kolejność wykonania, bo w takim kompleksie może wystąpić konieczność współdzielenia zasobów między operacjami.

Szukamy rozdziału zasobów, który minimalizuje czas wykonania operacji. Należy wyznaczyć takie $u^* \in D_u$, aby

$$u^* = \arg \min_{u \in D_u} T$$

Dane	Ograniczenia	1
$N = \{1, 2, \dots, i, \dots, n\}$ — Zbiór operacji w kompleksie n — Liczba operacji i — Indeks operacji T_i — Czas wykonania i-tej operacji - Model struktury kompleksu operacji $\rho \subset N \times N$ — niezależnych $\rho \subset N \times N$ — zależnych a_i — Parametr lub wektor parametrów dodatkowo charakteryzujących i-tę operację $T_i = \gamma_i(u_i, a_i)$ — Model operacji, γ jest: - Ciągle i ściśle malejące ze względu na u_i - Nieujemne $\lim_{u_i \rightarrow 0} \gamma_i(u_i, a_i) = +\infty, \lim_{u_i \rightarrow +\infty} \gamma_i(u_i, a_i) = 0$	$\forall i \in \{1, 2, \dots, n\} : u_i \geq 0$ Przydzielamy nieujemne wartości $\sum_{i=1}^n u_i \leq U$ Nie przekraczamy globalnej ilości zasobów	
	Przykład struktury $\rho \subset N \times N$ Graf Macierz incydencji $\kappa = \begin{bmatrix} 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$	
Zmienne decyzyjne $u = [u_1 \quad u_2 \quad \dots \quad u_n]^T$ Wektor zmiennych decyzyjnych U — Globalna ilość zasobów D_u — Zbiór dopuszczalnych zmiennych decyzyjnych	Kryterium oceny Czas wykonania operacji $T = \max_i \bar{t}_i - \min_i \underline{t}_i = \max_i (t_i + T_i) = \Phi(T_1, T_2, \dots, T_n)$ Gdzie Φ to odwzorowanie zależne od struktury kompleksu operacji. Dla struktury równoległej, czyli dla kompleksu operacji niezależnych $\Phi = \max(\cdot), T = \max(T_1, T_2, \dots, T_n)$	

System podejmowania decyzji, w tym obiekt podejmowania decyzji:



Gdzie $y \equiv T$ w postaci $\min T$

¹Nie zgadniecie kto to napisał

1.2 Szeregowanie zadań niezależnych i niepodzielnych, minimalizacja długości uszeregowania $R||C_{\max}$

Celem szeregowania zadań jest wyznaczenie dla każdego zadania maszyny wykonującej zadanie oraz czasu rozpoczęcia wykonywania zadania tak, aby zminimalizować wybrane (czasowe) kryterium jakości szeregowania.

Wyróżniamy 2 rodzaje maszyn:

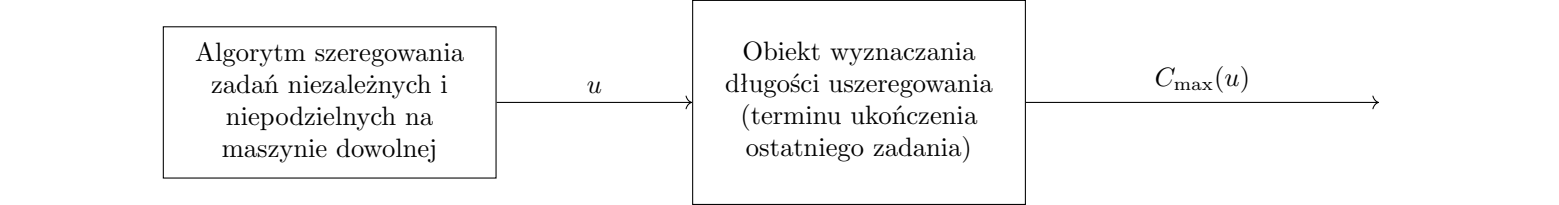
- Równoległe (uniwersalne)
- Dedykowane (specjalizowane)

Należy wyznaczyć macierz u^* , która minimalizuje długość uszeregowania

$$u^* = \arg \min_u C_{\max}(u)$$

Dane	Ograniczenia
• $M = \{M_1, M_2, \dots, M_i, \dots, M_m\}$ — Zbiór maszyn dowolnych • $J = \{J_1, J_2, \dots, J_j, \dots, J_n\}$ — Zbiór zadań • $p_j = [p_{1j} \ p_{2j} \ \dots \ p_{ij} \ \dots \ p_{mj}]^T$, gdzie p_{ij} to Czas wykonania j-tego zadania przez i-tę maszynę	• $\forall j \in \{1, 2, \dots, n\} : \sum_{i=1}^m u_{ij} = 1$ Każde zadanie musi być wykonane dokładnie raz • $u_{ij} \in \{0, 1\}$
Zmienne decyzyjne	Kryterium oceny
• $u_{ij} = \begin{cases} 1 & \text{Jeśli zadanie } i \text{ jest wykonywane przez maszynę } j \\ 0 & \text{wpp} \end{cases}$ • $u = [u_{ij}]_{i \in \{1, 2, \dots, n\}}$	Długość uszeregowania (termin skończenia ostatniego zadania) $C_{\max}(u) = \max(C_1, C_2, \dots, C_n) = \max(\overline{C_1}, \overline{C_2}, \dots, \overline{C_n}) =$ $= \max\left(\sum_{j=1}^n p_1 u_{1j}, \sum_{j=1}^n p_2 u_{2j}, \dots, \sum_{j=1}^n p_m u_{mj}\right) =$ $= \max_i \sum_{j=1}^n p_{ij} u_{ij}$

System podejmowania decyzji, w tym obiekt podejmowania decyzji:



1.3 Problem przepływowy z kryterium $C_{\max}(F||C_{\max})$

Problem szeregowania zadań na maszynach dedykowanych

- Każde zadanie składa się z takiej samej liczby operacji równej liczbie maszyn
- Przyporządkowanie maszyn do operacji jest dane
- Jest ustalona kolejność wykonywania operacji w ramach zadania, która jest taka sama dla każdego zadania

Rodzaje problemów przepływowych:

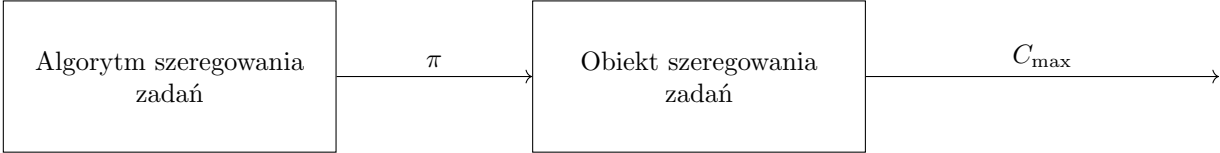
- Permutacyjne (każda maszyna przetwarza zadania w takiej samej kolejności)
- Niepermutacyjne
- Bez buforów albo z buforami (ograniczone albo nieograniczone)

Należy wyznaczyć permutację zadań, która minimalizuje czas ich wykonania

$$\pi^* = \arg \min_{\pi \in \Pi} C_{\max}(\pi)$$

Dane	Założenia	2
• $M = \{M_1, M_2, \dots, M_i, \dots, M_m\}$ — Zbiór maszyn • $J = \{J_1, J_2, \dots, J_j, \dots, J_n\}$ — Zbiór zadań • $J_j = (O_1, O_2, \dots, O_{ij}, \dots, O_{mj})$ Ciąg operacji j-tego zadania • $p = [p_{ij}]_{i \in \{1, 2, \dots, m\}, j \in \{1, 2, \dots, n\}}$, gdzie p_{ij} to Czas wykonania i-tej operacji zadania j	• Nie ma ograniczeń kolejnościowych między operacjami należącymi do różnych zadań • Każda maszyna w jednej chwili może wykonywać tylko jedno zadanie • Każde zadanie w jednej chwili może być wykonywane tylko na jednej maszynie • Rozpatrujemy problem permutacyjny bez buforów i z kryterium długość uszeregowania	
Zmienne decyzyjne	Kryterium oceny	
• Dowolna permutacja zadań należących do J czyli $\pi = (\pi(1), \pi(2), \dots, \pi(k), \dots, \pi(n))$ gdzie $\pi(k) \in \{1, 2, \dots, n\}$ to numer zadania wykonywanego jako k-te z kolei • $\Pi, \Pi = n!$ — zbiór wszystkich n-elementowych permutacji	• $C_{i, \pi(1)} = \sum_{l=1}^i p_{l, \pi(1)}$ Terminy zakończenia operacji pierwszego zadania • $C_{1, \pi(k)} = \sum_{l=1}^k p_{1, \pi(l)}$ Terminy zakończenia operacji wykonywanych przez pierwszą maszynę $i = 2, 3, \dots, m, k = 2, 3, \dots, n$ $C_{i, \pi(k)} = \max [C_{i-1, \pi(k)}, C_{i, \pi(k-1)}] + p_{i, \pi(k)}$ $C_{\max}(\pi) = C_{m, \pi(n)} = \max [C_{m-1, \pi(n)}, C_{m, \pi(n-1)}] + p_{m, \pi(n)}$	

System podejmowania decyzji, w tym obiekt podejmowania decyzji:



²Polecam tego pdfa, sporo wyjaśnia ładnie

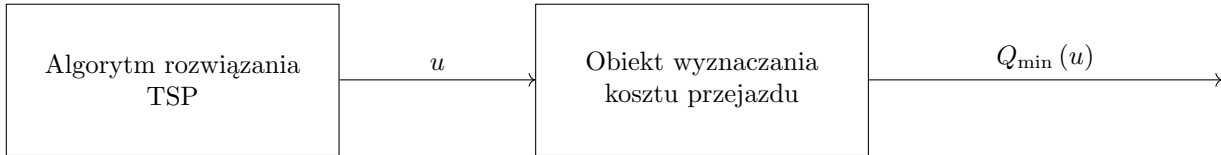
1.4 Problem komiwojażera

Travelling Salesman Problem, zadanie optymalizacji dyskretnej, problem silnie NP-trudny.

Szukamy trasy komiwojażera w postaci cyklu Hamiltona w grafie ważonym reprezentującym tę trasę (wspólny początek i koniec, każde miasto odwiedzany tylko raz). Cykl Hamiltona spełnia kryterium

$$u^* = \arg \min_u Q(u)$$

Dane	Ograniczenia
• $N = \{1, 2, \dots, n\}$ — Zbiór miast o liczebności n • $i, j \in \{1, 2, \dots, n\}$ — Indeksy miast • c_{ij} — Koszt podróży między miastami i oraz j • $C = [c_{ij}]_{i,j \in \{1,2,\dots,n\}}$ Koszt podróży między miastami i oraz j	• $\forall j \in \{1, 2, \dots, n\} : \sum_{i=1}^n u_{ij} = 1$ Komiwojażer musi dotrzeć do każdego miasta dokładnie raz • $\forall i \in \{1, 2, \dots, n\} : \sum_{j=1}^n u_{ij} = 1$ Komiwojażer musi opuścić każde miasto dokładnie raz • $\forall S \subset N : \sum_{i \in S} \sum_{j \in N-S} u_{ij} \geq 1$ Zapobieganie podcyklom
Zmienne decyzyjne	Kryterium oceny
• $u_{ij} = \begin{cases} 1 & \text{Jeśli komiwojażer przemieszcza się z miasta } i \text{ do } j \\ 0 & \text{wpp} \end{cases}$ • $u = [u_{ij}]_{i,j \in \{1,2,\dots,n\}}$	Suma kosztów przejazdów $y \triangleq Q(u) = \sum_{i=1}^n \sum_{j=1}^n c_{ij} u_{ij}$
System podejmowania decyzji, w tym obiekt podejmowania decyzji:	



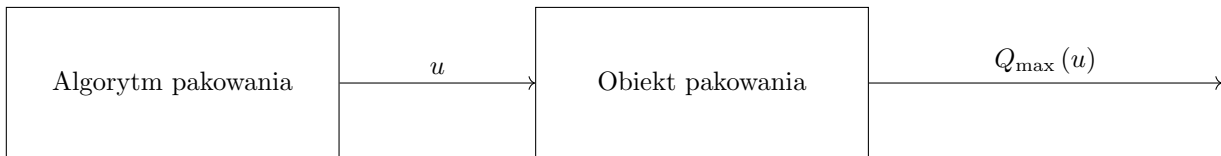
1.5 Problem plecakowy 0-1

0-1 Knapsack Problem, problem NP-trudny.

Szukamy takich przedmiotów do umieszczenia które zapewnią największą wartość przedmiotów przy nieprzekroczeniu objętości plecaka. Załadowanie plecaka u^* maksymalizujące kryterium

$$u^* = \arg \max_u Q(u)$$

Dane	Ograniczenia
• V — Rozmiar plecaka • n — Liczba elementów, które mogą być włożone do plecaka • i — Indeks elementu • v_i — Rozmiar i-tego elementu • w_i — Waga i-tego elementu	• $\forall i \in \{1, 2, \dots, n\} : u_i \in \{0, 1\}$ • $\sum_{i=1}^n v_i u_i \leq V$ Nieprzekraczanie pojemności plecaka
Zmienne decyzyjne	Kryterium oceny
• $u_i = \begin{cases} 1 & \text{Jeśli } i\text{-ty element włożono do plecaka} \\ 0 & \text{wpp} \end{cases}$ • $u = [u_i]_{i \in \{1,2,\dots,n\}}^T$	Wartość elementów umieszczonych w plecaku $Q(u) = \sum_{i=1}^n w_i u_i$
System podejmowania decyzji, w tym obiekt podejmowania decyzji:	



1.6 Problemy rozmieszczenia UFLP, UFLP i p-median

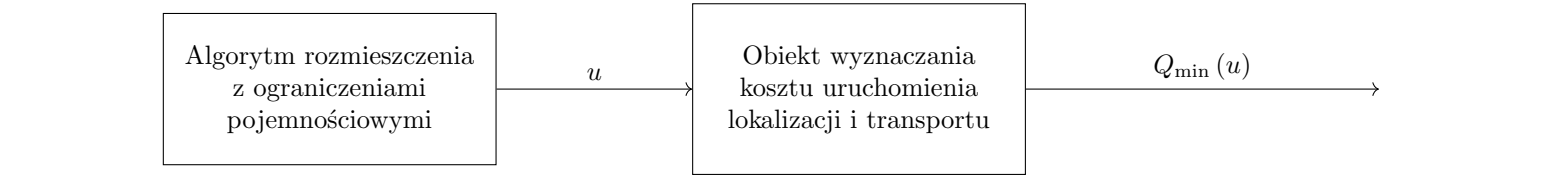
1.6.1 Capacitated Facility Location Problem

Dla danego zbioru klientów, z danego zbioru lokalizacji należy wybrać podzbiór lokalizacji gotowych do obsługi klientów, tak aby minimalizować łączny koszt uruchomienia lokalizacji i transportu

$$u^* = \arg \min_u Q(u)$$

Dane	Ograniczenia
$L = \{1, 2, \dots, i, \dots, n\}$ Zbiór n lokalizacji, i to bieżący indeks lokalizacji $C = \{1, 2, \dots, j, \dots, m\}$ Zbiór m klientów, j to bieżący indeks klienta f_i — Koszt uruchomienia lokalizacji i c_{ij} — Koszt transportu z lokalizacji i do klienta j d_j — Popyt klienta j v_i — Pojemność lokalizacji i (ilość dobra generowanego w lokalizacji i)	$\forall j \in \{1, 2, \dots, m\} : \sum_{i=1}^n y_{ij} = 1$ Każdy klient jest obsługiwany $\forall i \in \{1, 2, \dots, n\} : \sum_{j=1}^m d_j y_{ij} \leq v_i x_i$ Nie można zaspokoić popytu klienta z nieuruchomionej lokalizacji $\forall i \in \{1, 2, \dots, n\} : x_i \in \{0, 1\}$ $0 \leq y_{ij} \leq 1$
Zmienne decyzyjne	Kryterium oceny
$x_i = \begin{cases} 1 & \text{Jeśli } i\text{-ta lokalizacja jest uruchomiona} \\ 0 & \text{wpp} \end{cases}$ y_{ij} — Część popytu klienta j zaspokojona z lokalizacji i $u \triangleq (x_i, y_{ij}, i = \overline{1, n}, j = \overline{1, m})$ — Decyzja	Łączny koszt uruchomienia lokalizacji i transportu dla wybranego podzbioru lokalizacji $Q(u) = \min_{u \equiv (x, y)} \sum_{i=1}^n \sum_{j=1}^m c_{ij} y_{ij} + \sum_{i=1}^n f_i x_i$

System podejmowania decyzji, w tym obiekt podejmowania decyzji:



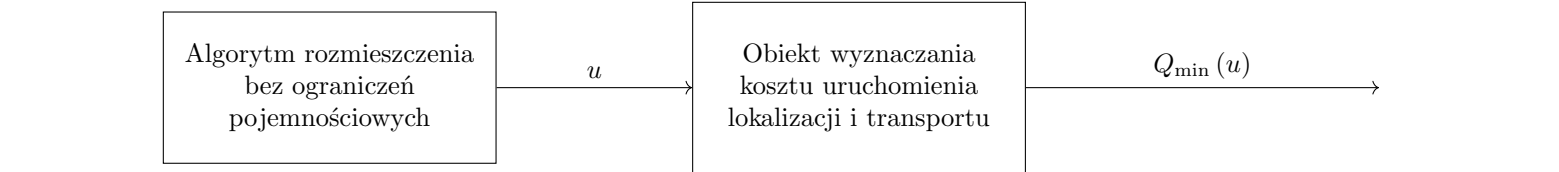
1.6.2 Uncapacitated Facility Location Problem

Dla danego zbioru klientów, z danego zbioru lokalizacji należy wybrać podzbiór lokalizacji gotowych do obsługi klientów, tak aby minimalizować łączny koszt uruchomienia lokalizacji i transportu

$$u^* = \arg \min_u Q(u)$$

Dane	Ograniczenia
$L = \{1, 2, \dots, i, \dots, n\}$ Zbiór n lokalizacji, i to bieżący indeks lokalizacji $C = \{1, 2, \dots, j, \dots, m\}$ Zbiór m klientów, j to bieżący indeks klienta f_i — Koszt uruchomienia lokalizacji i c_{ij} — Koszt transportu z lokalizacji i do klienta j $v_i = \infty$ Każda lokalizacja generuje nieskończone zasoby i całkowity popyt klienta jest zaspokajany w najbliższej lokalizacji	$\forall j \in \{1, 2, \dots, m\} : \sum_{i=1}^n z_{ij} = 1$ Każdy klient jest obsługiwany $\forall i \in \{1, 2, \dots, n\}, j \in \{1, 2, \dots, m\} : z_{ij} \leq x_i$ Nie można zaspokoić popytu klienta z nieuruchomionej lokalizacji $\forall i \in \{1, 2, \dots, n\}, j \in \{1, 2, \dots, m\} : x_i, z_{ij} \in \{0, 1\}$
Zmienne decyzyjne	Kryterium oceny
$x_i = \begin{cases} 1 & \text{Jeśli } i\text{-ta lokalizacja jest uruchomiona} \\ 0 & \text{wpp} \end{cases}$ $z_{ij} = \begin{cases} 1 & \text{Jeśli } j\text{-ty klient jest obsługiwany przez } i\text{-tą lokalizację} \\ 0 & \text{wpp} \end{cases}$ $u \triangleq (x_i, z_{ij}, i = \overline{1, n}, j = \overline{1, m})$ — Decyzja	Łączny koszt uruchomienia lokalizacji i transportu dla wybranego podzbioru lokalizacji $Q(u) = \min_{u \equiv (x, y)} \sum_{i=1}^n \sum_{j=1}^m c_{ij} z_{ij} + \sum_{i=1}^n f_i x_i$

System podejmowania decyzji, w tym obiekt podejmowania decyzji:



1.6.3 P-median

Problem jest NP-trudny (nie jest silnie NP-trudny). Problem *p-median* różni się od *UFLP* w dwóch aspektach:

- Nie ma kosztów inwestycyjnych
- Jest wymagana liczba p uruchomionych lokalizacji

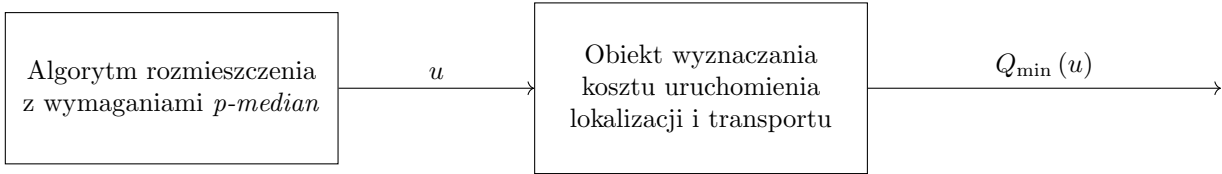
Problem także modeluje zagadnienie znajdowania minimalno-kosztowej klasteryzacji

Dla danego zbioru klientów, z danego zbioru lokalizacji należy wybrać podzbiór lokalizacji gotowych do obsługi klientów, tak aby minimalizować łączny koszt uruchomienia lokalizacji i transportu

$$u^* = \arg \min_u Q(u)$$

Dane	Ograniczenia	3
• $L = \{1, 2, \dots, i, \dots, n\}$ Zbiór n lokalizacji, i to bieżący indeks lokalizacji • $p, p \leq n$ Liczba lokalizacji do uruchomienia spośród n dostępnych • $S, S \subseteq L$ — p-elementowy podzbiór L • $C = \{1, 2, \dots, j, \dots, m\}$ Zbiór m klientów, j to bieżący indeks klienta • c_{ij} — Koszt transportu z lokalizacji i do klienta j • $v_i = \infty$ Każda lokalizacja generuje nieskończone zasoby i całkowity popyt klienta jest zaspokajany w najbliższej lokalizacji	• $\forall j \in \{1, 2, \dots, m\} : \sum_{i=1}^n z_{ij} = 1$ Każdy klient jest obsługiwany • $\forall i \in \{1, 2, \dots, n\}, j \in \{1, 2, \dots, m\} : z_{ij} \leq x_i$ Nie można zaspokoić popytu klienta z nieuruchomionej lokalizacji • $\sum_{i=1}^n x_i = p$ — Wybieramy p lokalizacji • $\forall i \in \{1, 2, \dots, n\}, j \in \{1, 2, \dots, m\} : x_i, z_{ij} \in \{0, 1\}$	
Zmienne decyzyjne	Kryterium oceny	
• $x_i = \begin{cases} 1 & \text{Jeśli } i\text{-ta lokalizacja jest uruchomiona} \\ 0 & \text{wpp} \end{cases}$ • $z_{ij} = \begin{cases} 1 & \text{Jeśli } j\text{-ty klient jest obsługiwany przez } i\text{-tą lokalizację} \\ 0 & \text{wpp} \end{cases}$ • $u \triangleq (x_i, z_{ij}, i = \overline{1, n}, j = \overline{1, m})$ — Decyzja	Łączny koszt uruchomienia lokalizacji i transportu dla wybranego podzbioru lokalizacji $Q(u) = \min_{u \equiv S \subseteq L} \sum_C \min_{i \in S} c_{ij}$	

System podejmowania decyzji, w tym obiekt podejmowania decyzji:



³Część ograniczeń poprawiłem na podstawie tego ślicznego pdfa

Rozdział 2

Wybrane metody i algorytmy rozwiązywania zagadnień podejmowania decyzji

2.1 Wieloetapowe podejmowanie decyzji

2.1.1 Sformułowanie problemu wieloetapowego podejmowania decyzji

Dane	Do wyznaczenia
• x_0 • $x_{n+1} = f(x_n, u_n)$ • $\varphi(x_n, u_{n-1})$	$\bar{u}_n \triangleq (u_0, u_1, \dots, u_n, \dots, u_{N-1})$ Tak, aby minimalizować $Q_N(\bar{u}_N) = \sum_{n=1}^N \varphi(x_n, u_{n-1})$ $\Downarrow$ $\bar{u}_N^* = \arg \min_{\bar{u}_N} \sum_{n=1}^N \varphi(x_n, u_{n-1})$

2.1.2 Zasada optymalności Bellmana

Jeśli optymalny proces decyzyjny **przerwiemy w chwili** n w stanie x_n , a następnie ponownie go podejmiemy przyjmując x_n za stan początkowy, to **dalszy ciąg procesu decyzyjnego** (tzn. począwszy od n) **będzie taki sam**, jaki byłby, gdyby nie przerywać procesu, który się rozpoczął w x_0 .

Innymi słowy, stan x_n w pełni determinuje dalszy proces decyzyjny, a w x_n jest zawarta pełna informacja o poprzednich decyzjach

2.1.3 Programowanie dynamiczne

$$Q_N(\bar{u}_N) = \sum_{n=1}^N \varphi(x_n, u_{n-1}) = \sum_{n=1}^N \varphi(f(x_{n-1}, u_{n-1}), u_{n-1}) = \sum_{n=1}^N g(x_{n-1}, u_{n-1}) = \sum_{n=0}^{N-1} g(x_n, u_n)$$

$$V_N(x_0) = \min_{\bar{u}_N} Q_N(\bar{u}_N)$$

Przebieg wstecz

- $n = N - 1$

$$V_1(x_{N-1}) = \min_{u_{N-1}} g(x_{N-1}, u_{N-1}) \longrightarrow u_{N-1}^* = \Psi_{N-1}(x_{N-1})$$
- $n = N - 2$

$$V_2(x_{N-2}) = \min_{u_{N-2}, u_{N-1}} [g(x_{N-2}, u_{N-2}) + g(x_{N-1}, u_{N-1})] = \min_{u_{N-2}} \left[g(x_{N-2}, u_{N-2}) + \min_{u_{N-1}} g(x_{N-1}, u_{N-1}) \right] =$$

$$= \min_{u_{N-2}} [g(x_{N-2}, u_{N-2}) + V_1(x_{N-1} = f(x_{N-2}, u_{N-2}))] \longrightarrow$$

$$\rightarrow u_{N-2}^* = \Psi_{N-2}(x_{N-2})$$
- $\vdots$
- $n = 0$

$$V_N(x_0) = \min_{u_0} [g(x_0, u_0) + V_{N-1}(x_1 = f(x_0, u_0))] \longrightarrow u_0^* = \Psi_0(x_0)$$

Przebieg w przód

- $u_0^* = \Psi_0(x_0)$
- $u_1^* = \Psi_1(x_1) \longleftarrow x_1 = f(x_0, u_0^*)$
- $u_2^* = \Psi_2(x_2) \longleftarrow x_2 = f(x_1, u_1^*)$
- $\vdots$
- $u_{N-1}^* = \Psi_{N-1}(x_{N-1}) \longleftarrow x_{N-1} = f(x_{N-2}, u_{N-2}^*)$

2.2 Wielokryterialne podejmowanie decyzji

2.2.1 Wielokryterialne podejmowanie decyzji

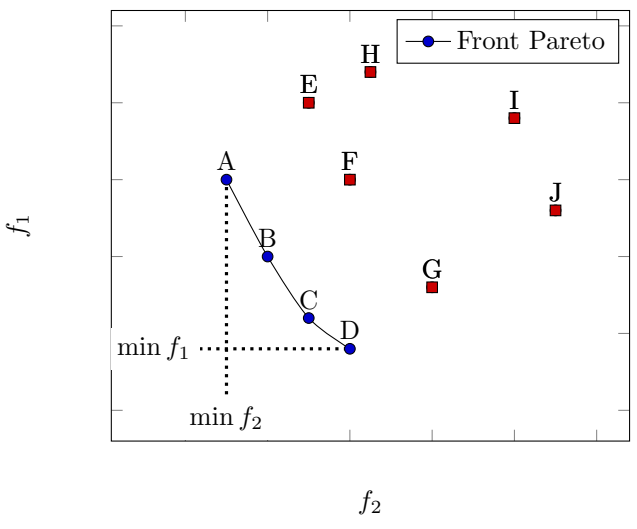
- $u \in U$ — Decyzja należąca do zbioru decyzji dopuszczalnych, np. zmienna skalarna, wektorowa
- $y = [y^{(1)} \ y^{(2)} \ \dots \ y^{(i)} \ \dots \ y^{(k)}]^T$ — Wektor k kryteriów (ocen decyzji u)
- $y^{(i)} = f^{(i)}(u)$
- $y \in Y, Y$ — Przestrzeń wektorów kryteriów (ocen)
- $F : U \rightarrow Y$ — Odwzorowanie (w szczególności funkcja) przyporządkowujące każdej decyzji u wektor ocen (kryteriów) y (zakładamy, że decyzje u są całkowicie jednoznacznie reprezentowane przez oceny y)

2.2.2 Front Pareto

W przestrzeni ocen Y definiujemy **relację dominacji** (częściowego porządku). Mówimy, że ocena y dominuje ocenę $\hat{y}$, jeśli

$$\begin{cases} \forall i \in \overline{1, k} : y^{(i)} \leq \hat{y}^{(i)} \\ \exists i \in \overline{1, k} : y^{(i)} < \hat{y}^{(i)} \end{cases}$$

Front Pareto w przestrzeni ocen tworzą **wszystkie oceny dominujące**, tzn. żadna ocena należąca do frontu Pareto nie jest zdominowana przez inną ocenę z Y



Problem wielokryterialnego podejmowania decyzji (optymalizacji wielokryterialnej lub optymalizacji wektorowej) polega faktycznie na wyznaczeniu zbioru Pareto ocen, a w konsekwencji odpowiadającego mu zbioru decyzji
O decyzjach u odpowiadających zbiorowi Pareto w Y mówimy, że są **Pareto optymalne**

2.2.3 Skalaryzacja, sprowadzanie do problemów jednokryterialnych

1. Określenie metakryterium (funkcji kryteriów cząstkowych)

$$G(u) \triangleq G[f^{(1)}(u), f^{(2)}(u), \dots, f^{(k)}(u)]$$

- (a) Ważona suma kryteriów jednorodnych

$$G(u) = \sum_{i=1}^k w_i f^{(i)}(u), w_i \geq 0, \sum_{i=1}^k w_i = 1$$

- (b) Ważona suma kryteriów niejednorodnych

$$G(u) = \sum_{i=1}^k w_i \frac{f^{(i)}(u)}{f^{(i)*}}, w_i \geq 0, \sum_{i=1}^k w_i = 1$$

gdzie $f^{(i)*} = \min_{u \in U} f^{(i)}(u)$

2.2.4 Optymalizacja celu głównego

i -te kryterium wybieramy jako cel główny

$$\min_{u \in U} f^{(i)}(u)$$

s.t.

$$\forall l \in \{1, 2, \dots, k\}, l \neq i : f^{(l)}(u) \leq b_l$$

gdzie b_l to zadane wartości

2.2.5 Minimalizacja odległości od punktu referencyjnego

Punkt referencyjny (niekoniecznie należący do Y)

$$\bar{y} \triangleq [\bar{f}^{(1)}(u) \ \bar{f}^{(2)}(u) \ \dots \ \bar{f}^{(k)}(u)]^T$$

Wówczas $\min_{u \in U} \|y - \bar{y}\|$, a w szczególności

$$\bar{y} \triangleq [f^{(1)*}(u) \ f^{(2)*}(u) \ \dots \ f^{(k)*}(u)]^T$$

2.2.6 Metoda AHP

Analytical Hierarchy Process

Rozważamy dalej szczególnie przypadek n wariantów decyzji- Hierarchiczna struktura problemu decyzyjnego w metodzie AHP nych, tzn.

$$u \in U \triangleq \{u_1, u_2, \dots, u_j, \dots, u_n\}$$

Wartości kryteriów $f^{(i)}(u)$ nie muszą być liczbowe
Założenia:

- Brak analitycznej postaci oceny wariantów decyzyjnych
- Skończona liczba wariantów decyzyjnych

Rangi liczbowe stosowane przy porównaniach kryteriów i wariantów

Ocena słowna	Ocena liczbowa
Równie preferowane	1
	2
Nieznacznie preferowane	3
	4
Silnie preferowane	5
	6
Bardzo silnie preferowane	7
	8
Nadzwyczaj preferowane	9

Macierz porównań kryteriów

$$M^{(0)} = \left[m_{hl}^{(0)} \right]_{h,l \in \{1,2,\dots,k\}}$$

$$\forall h \in \{1,2,\dots,k\} : m_{hh}^{(0)} = 1$$

$$\forall h,l \in \{1,2,\dots,k\} : m_{hl}^{(0)} = \frac{1}{m_{lh}^{(0)}}$$

$$\forall h,l,p \in \{1,2,\dots,k\} : m_{hl}^{(0)} = m_{hp}^{(0)} \cdot m_{pl}^{(0)}$$

1. Normalizacja macierzy porównań

$$\overline{M}^{(0)} = \left[\overline{m}_{hl}^{(0)} \right]_{h,l \in \{1,2,\dots,k\}}$$

$$\forall h,l \in \{1,2,\dots,k\} : \overline{m}_{hl}^{(0)} = \frac{m_{hl}^{(0)}}{\sum_{h=1}^k m_{hl}^{(0)}} = \frac{m_{hl}^{(0)}}{c_l^{(0)}}$$

$$\forall i \in \{1,2,\dots,k\} : \overline{M}^{(i)} = \left[\overline{m}_{jl}^{(i)} \right]_{j,l \in \{1,2,\dots,n\}}$$

3. Obliczenie rankingu końcowego

$$\forall j \in \{1,2,\dots,n\} : r_j = \sum_{h=1}^k s_h^{(0)} \cdot s_j^{(h)}$$

Rozwiązanie w postaci ciągu decyzji (rankingu) tworzą decyzje uporządkowane w kolejności nierosnących wartości współczynników r_j

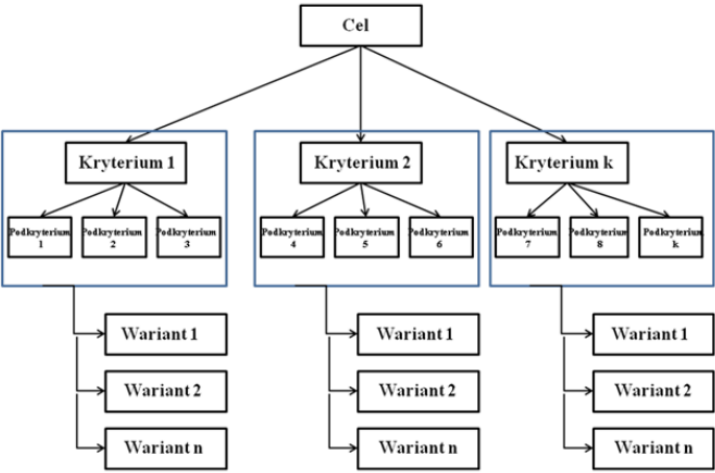
Zapewnienie spójności macierzy preferencji

Macierze są spójne jeśli wektory indywidualnych wskaźników preferencji są wektorami własnymi odpowiednich macierzy preferencji

$$M^{(0)} \cdot s^{(0)} = k \cdot s^{(0)} \quad M^{(i)} \cdot s^{(i)} = n \cdot s^{(i)}, i = 1,2,\dots,k$$

Praktyczne sprawdzanie spójności

$$CR = \frac{CI}{RI} \leq 0.1 \text{ — Wskaźnik spójności, } CI = \frac{\sum_{l=1}^k c_l^{(0)} \cdot s_l^{(0)} - k}{k - l}, \forall i \in \{1,2,\dots,k\} : RI = \frac{\sum_{l=1}^n c_l^{(i)} \cdot s_l^{(i)} - n}{n - 1}$$



Macierz porównań decyzji (wariantów decyzyjnych)

$$\forall i \in \{1,2,\dots,k\} : M^{(i)} = \left[m_{jl}^{(i)} \right]_{j,l \in \{1,2,\dots,n\}}$$

$$\forall j \in \{1,2,\dots,n\}, i \in \{1,2,\dots,k\} : m_{jj}^{(i)} = 1$$

$$\forall j,l \in \{1,2,\dots,n\}, i \in \{1,2,\dots,k\} : m_{jl}^{(i)} = \frac{1}{m_{lj}^{(i)}}$$

$$\forall j,l,p \in \{1,2,\dots,n\}, i \in \{1,2,\dots,k\} : m_{jl}^{(i)} = m_{jp}^{(i)} \cdot m_{pl}^{(i)}$$

2. Obliczenie średnich wartości elementów macierzy znormalizowanych w każdym wierszu

$$s^{(0)} = \begin{bmatrix} s_1^{(0)} & s_2^{(0)} & \dots & s_h^{(0)} & \dots & s_k^{(0)} \end{bmatrix}^T$$

$$\forall h \in \{1,2,\dots,k\} : s_h^{(0)} = \frac{1}{k} \sum_{l=1}^k \overline{m}_{hl}^{(0)}$$

$$s^{(i)} = \begin{bmatrix} s_1^{(i)} & s_2^{(i)} & \dots & s_j^{(i)} & \dots & s_n^{(i)} \end{bmatrix}^T$$

$$\forall j \in \{1,2,\dots,n\}, i \in \{1,2,\dots,k\} : s_j^{(i)} = \frac{1}{n} \sum_{l=1}^n \overline{m}_{jl}^{(i)}$$

Im większa wartość $s_j^{(i)}$, tym wyższa pozycja wariantu j w rankingu decyzji (tylko ze względu na kryterium i)

Analogicznie: Im większa wartość $s_h^{(0)}$, tym większe znaczenie kryterium h

2.2.7 Metoda ELECTRE I

1. Ustal wagę kryterium w_i , odzwierciedlającą wagność przypisaną mu przez decydenta $\sum_{i=1}^k w_i = 1$
2. Oblicz wartość relacji φ_i dla każdego kryterium i i dla każdej pary wariantów decyzyjnych (u_j, u_l)

$$\varphi_i(u_j, u_l) = \begin{cases} 1 & f^{(i)}(u_j) \geq f^{(i)}(u_l) \\ 0 & \text{wpp} \end{cases}$$

3. Oblicz wartość współczynnika zgodności $c(u_j, u_l)$ dla każdej pary wariantów decyzyjnych (u_j, u_l)

$$c(u_j, u_l) = \sum_{i=1}^k w_i \varphi_i(u_j, u_l)$$

4. Wyznacz zbiór zgodności $C_s = \{(u_j, u_l) : c(u_j, u_l) \geq s\}$, gdzie s to próg zgodności podany przez decydenta, najczęściej $\frac{1}{2} \leq s \leq 1$. Zbiór zgodności C_s zawiera te pary decyzji, w których pierwsza z decyzji jest średnio lepsza od drugiej więcej niż wartość progu zgodności s , przy czym uśrednienie jest dokonywane po wszystkich kryteriach z uwzględnieniem ich wagności dla decydenta.

5. Wyznacz zbiór niezgodności

$$D_v = \left\{ (u_j, u_l) \in C_s : \exists f^{(i)} : f^{(i)}(u_l) > f^{(i)}(u_j) + v_i \right\}$$

gdzie v_i - próg weta podany przez decydenta

Zbiór niezgodności D_v zawiera te pary decyzji, w których druga decyzja z pary decyzji jest lepsza od pierwszej o wartość większą od progu weta v_i (lub równoważnie — pierwsza jest gorsza od drugiej o więcej niż v_i).

6. Wyznacz relację przewyższania S jako $S(s, v) = C_s - D_v$

7. Utwórz ciąg

$$(U_1, U_2, \dots, U_m, \dots, U_N) \triangleq (U_m)_{m \in \{1, 2, \dots, n\}}, U_m \subseteq U, \bigcup_{m=1}^N U_m = U, U_{m_1} \cap U_{m_2} = \emptyset$$

N podzbiorów wariantów decyzyjnych w taki sposób, że elementy pary decyzji będących w relacji S należą do sąsiednich podzbiorów z ciągu w kolejności malejącej relacji przewyższania.

2.3 Metaheurystyki SA i TS

Rozważmy ogólnie zadanie optymalizacji (minimalizacji) $\min_{u \in U} f(u)$ (uproszczenie zapisu $U \equiv D_u$)

2.3.1 Symulowane wyżażanie

Inspiracja

Zjawisko zachodzące w gazie doskonałym, gdzie za pomocą rozkładu Boltzmanna opisuje się rozkład liczby cząsteczek o danej energii. W krystalografii **wyżarzaniem** nazywa się proces usuwania naprężeń w ciele stałym o strukturze krystalicznej.

Proces ten składa się z 2 faz:

1. Zwiększenie temperatury, aż do osiągnięcia wartości przy której ciało zaczyna przechodzić w stan ciekły, energie wew. cząsteczek są bardzo duże
2. **Powolne** zmniejszanie temperatury tak, by cząsteczki **samoczynnie** osiągnęły stan podstawowy — o minimalnej energii

Pojęcia wstępne

- u_i — Bieżące rozwiązanie dopuszczalne ($u_i \in U$)
- i — Numer rozwiązania dopuszczalnego
- $N : U \longrightarrow 2^{|U|}$ — Odwzorowanie generujące sąsiedztwo (struktura sąsiedztwa).
Odwzorowanie N generuje dla każdego u_i zbiór $U_i \subset U$ rozwiązań w pewnym sensie *bliskich* u_i
- $u_j \in U_i$ — Rozwiązanie sąsiednie

Mechanizm generowania sąsiedztwa

Czyli sposób wyboru u_j z U_i .

- **Przejście** — działanie prowadzące do osiągnięcia j dla bieżącego rozwiązania i
 1. Generacja sąsiedztwa j rozwiązania i
 2. Akceptacja wygenerowanego rozwiązania dopuszczalnego $j \in U_i$

Pseudokod

SA

Oznaczenia

- | | |
|---|---|
| 1. zainicjuj (u_0, t_0) | • k — Numer iteracji |
| 2. $u_i := u_0, t_k := t_0, k := 0$ | • L_k — Liczba wygenerowanych przejść w k -tej iteracji |
| 3. Dopóki STOP (warunek stopu) | • t_k — Parametr kontrolny — temperatura |
| 4. Dla $k := 1$ do L_k wykonuj | |
| 5. Generuj u_j ze zbioru U_i | |
| 6. $\Delta f_{ij} := f(u_j) - f(u_i)$ | |
| 7. Jeśli $\Delta f_{ij} \leq 0$ to | |
| 8. $u_i := u_j$ | |
| 9. Wpp jeśli $\exp(-\frac{\Delta f_{ij}}{t_k}) > \text{rand}[0,1)$ to | |
| 10. $u_i := u_j$ | |
| 11. $k := k+1$ | |
| 12. uaktualnij t_k | |
| 13. uaktualnij L_k | |

2.3.2 Tabu search

Inspiracja

Fred Glover - wykorzystanie **pamięci krótko i długotrwałej** w przeszukiwaniu lokalnym, długotrwała miała służyć do zapamiętywania najbardziej atrakcyjnych elementów przestrzeni przeszukiwań, a krótkotrwała służyła do zapamiętywania ostatnich ruchów i ulegała nadpisaniu w kolejnych iteracjach przeszukiwania.

2.3.3 Pojęcia ogólne

W przypadku TS występuje dynamiczna zmiana sąsiedztwa danego rozwiązania, zbiór U_i nie jest z góry ustalony dla każdego u_i , ale może się zmieniać w zależności od dotychczasowego przebiegu algorytmu.

Występują 2 nowe pojęcia:

- **Lista tabu** — zawiera zestaw *zakazanych* rozwiązań dopuszczalnych u_i
- **Kryterium aspiracji** — Jest to mechanizm umożliwiający wcześniejsze usunięcie rozwiązania dopuszczalnego z TL albo wykonywanie elementu z TL bez jego usuwania z TL.

Ogólna postać TL	Szczególna postać TL
$TL = \{\forall l \in \{1, 2, \dots, L\} : (\text{atrybut}_l, \text{kadencja}_l)\}$	$TL = \{\forall l \in \{1, 2, \dots, L\} : u_l u_l \in U\}$
gdzie	Taka lista wprost zawiera <i>zakazane</i> rozwiązania dopuszczalne, a okres przebywania tych rozwiązań na TL jest stały. Po zapelnieniu TL przez L rozwiązań dopuszczalnych, rozwiązanie najdłużej przebywające na liście jest z niej usuwane (mechanizm kolejki FIFO).
• atrybut_l — Warunek (dane) niezbędne do wykonania ruchu l — para (u_i, u_j) (rozwiązanie aktualne, rozwiązanie wybrane z sąsiedztwa) • kadencja_l — Liczba iteracji, przez które atrybut_l znajduje się na liście	

Pseudokod

TS

1. zainicjuj u_0
2. $u_i := u_0, f_{\text{best}} := f(u_0), u_{\text{best}} := u_0, TL := \emptyset$
3. Dopóki STOP (warunek stopu)
4. $u_i := \arg \min_{u_l \in \tilde{U}_i} f(u_l)$
5. Jeśli $f(u_i) < f_{\text{best}}$ to
6. $f_{\text{best}} := f(u_i)$
7. $u_{\text{best}} := u_i$
8. umieść u_i w TL i usuń z TL najstarsze rozwiązanie jeśli jest to konieczne

2.4 Algorytmy rozwiązywania problemów alokacji zasobów w kompleksie operacji niezależnych i zależnych

2.4.1 Operacje niezależne

Algorytm rozwiązywania problemu opiera się na dwóch przesłankach

- Najkrótszy czas wykonywania wszystkich operacji T^* uzyskamy wówczas, gdy wszystkie operacje będą trwały jednakowo długo
- Należy wykorzystać wszystkie dostępne zasoby U

$$\begin{cases} \forall i \in \{1, 2, \dots, n\} : \gamma_i(u_i^*; a_i) = T^* \\ \sum_{i=1}^n u_i^* = U \end{cases}$$

Algorytm składa się z trzech kroków:

1. Wyznacz u_i^* z n pierwszych równań za pomocą funkcji odwrotnych, czyli $\forall i \in \{1, 2, \dots, n\} : u_i^* = \gamma_i^{-1}(T^*; a_i)$
2. Rozwiąż równanie $\sum_{i=1}^n \gamma_i^{-1}(T^*; a_i) = U$ i wyznacz $T^* = G(U, a_1, a_2, \dots, a_n)$
3. Wstaw G do γ_i^{-1} i oblicz $u_i^* = \gamma_i^{-1}(G(U, a_1, a_2, \dots, a_n); a_i) \triangleq \eta_i(U, a_1, a_2, \dots, a_n)$
 $\forall i \in \{1, 2, \dots, n\} : \eta_i$ — algorytm rozdziału zasobów

Warunek stosowania algorytmu — istnienie γ_i^{-1} oraz G

Np. rozdział pamięci RAM w systemie wieloprocesorowym

- U — Globalna ilość pamięci
- n — Liczba procesorów
- v_i — wielkość zadania obliczeniowego przydzielonego procesorowi i .
Operacja i polega na wykonaniu przez procesor i przydzielonego zadania o wielkości v_i

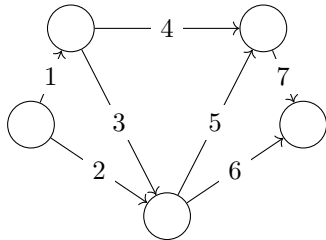
Im więcej pamięci RAM przydzielimy procesorowi, tym większa będzie szybkość jego działania i krótszy będzie czas wykonania tego zadania.

2.4.2 Operacje o dowolnej strukturze

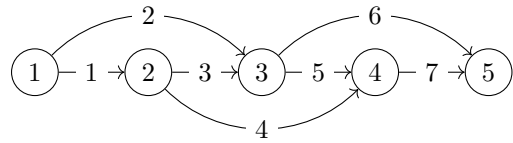
Algorytm:

1. Dekompozycja struktury kompleksu na przedziały
2. Rozwiązanie w przedziałach problemów alokacji dla operacji traktowanych jako niezależne, czyli tworzących kompleksy operacji niezależnych
3. Koordynacja rozwiązań wyznaczonych w kroku 2.

Dekompozycja



$$\begin{aligned}
 S_1 &= \{1, 2\} & S_2 &= \{2, 3, 4\} & S_3 &= \{4, 5, 6\} & S_4 &= \{6, 7\} \\
 Q_1 &= \{1\} & Q_2 &= \{1, 2\} & Q_3 &= \{2\} & Q_4 &= \{2, 3\} \\
 Q_5 &= \{3\} & Q_6 &= \{3, 4\} & Q_7 &= \{4\}
 \end{aligned}$$



Oznaczenia:

- l — Liczba zdarzeń (liczba wierzchołków w grafie reprezentującym kompleks operacji)
- j — Indeks przedziału, $j \in \{1, 2, \dots, l-1\}$
- S_j — Zbiór operacji należących do przedziału j
- Q_j — Zbiór indeksów przedziałów, w których jest wykonywana operacja i

Alokacja zasobów w j -tym przedziale

W tym kroku jest wykorzystywany algorytm dla kompleksu operacji niezależnych. Jest potrzebne uzupełnienie oznaczeń

- $\forall j \in \{1, 2, \dots, l-1\} : U_j \triangleq \{u_{ij} : i \in S_j\}$ — Zbiór wielkości zasobów u_{ij} przydzielonych do wszystkich operacji i wykonywanych w przedziale j
- x_{ij} — Rozmiar operacji i wykonywanej w przedziale j
- $X_j \triangleq \{x_{ij} : i \in S_j\}$ — Zbiór wszystkich rozmiarów operacji x_{ij} w j -tym przedziale
- $T_{ij} = \gamma_i(u_{ij}, x_{ij})$ — Czas wykonywania tej części operacji i , która jest wykonywana w przedziale j

Kryterium:

$$T^j = F_j(U_j, X_j) = \max_{i \in S_j} [\gamma_i(u_{ij}, x_{ij})]$$

gdzie U_j to zmienna decyzyjna, a X_j to stały parametr

Algorytm rozwiązania dla i -tego przedziału

$$\forall i \in S_j : u_{ij}^*(X_j) = \eta_j(X_j; U)$$

może być wyznaczony z układu równań

$$\begin{cases} \sum_{i \in S_j} u_{ij}^* = U \\ \gamma_i(u_{ij}^*; x_{ij}) = T^{j*} & i \in S_j \\ G_j(X_j; U) = T^{j*} & i \in S_j \end{cases}$$

Koordynacja

Celem tego kroku jest wyznaczenie X_j^* . W tym celu należy rozwiązać problem optymalizacji z takimi ograniczeniami:

Problem	Ograniczenia
$T(X_1^*, X_2^*, \dots, X_{l-1}^*) = \min_{X_1, X_2, \dots, X_{l-1}} \sum_{j=1}^{l-1} T^{j*}(X_j) = \min_{X_1, X_2, \dots, X_{l-1}} \sum_{j=1}^{l-1} G_j(X_j; U)$	$ \begin{cases} x_{ij} \geq 0 & \forall i \in S_j, j \in \{1, 2, \dots, l\} \\ \sum_{j \in Q_i} x_{ij} = v_i & \forall i \in \{1, 2, \dots, n\} \end{cases} $

Po rozwiązaniu należy wstawić X_j^* do $\eta_j(X_j; U)$ i obliczyć $u_{ij}^*(X_j^*)$

2.5 Algorytmy szeregowania zadań dla problemu $R||C_{\max}$ i dla problemu $F||C_{\max}$

2.5.1 Algorytm Ibarry

Algorytm przybliżony

Złożoność obliczeniowa
Aproksymowalność

$$C_{\max}^{\text{Ibarra}} \leq m C_{\max}^*$$
$$O(n)$$

- 1. Podstaw $T^i := 0$
- 2. Podstaw $u_{ij} := 0$
- 3. Dla $j = 1$ do n wykonaj
- 4. Znajdź najmniejszy indeks l maszyny dla którego

$$\forall i \in \{1, 2, \dots, m\}, i \neq l : T^l + p_{lj} \leq T^i + p_{ij}$$

i podstaw $T^l := T^l + p_{ij}$ oraz $u_{lj} := 1$

2.5.2 Algorytm Johnsona

Problem przepływowy dla maszyn dedykowanych, oraz dla $m = 2$

Algorytm przybliżony

Złożoność obliczeniowa

$O(n \log n)$

- 1. Utwórz ciąg S_1 dla tych zadań z J , dla których $p_{1j} \leq p_{2j}$, który jest posortowany w kolejności niemalejących czasów wykonania p_{1j}
- 2. Utwórz ciąg $S_2 = J - S_1$ pozostałych zadań z J , który jest posortowany w kolejności nierosnących czasów wykonania zadań p_{2j}
- 3. Scal ciągi S_1 i S_2 w jedną listę $(S_1 S_2)$ i wykonuj zadania w kolejności określonej przez tę listę

2.5.3 Algorytm NEH (Nawaz Ensore Ham)

Dla maszyn dedykowanych dla $m > 2$

Algorytm przybliżony

Złożoność obliczeniowa

$O(mn^2)$

- 1. Posortuj zadania zgodnie z nierosnącym porządkiem czasów ich wykonania na wszystkich maszynach, tzn. według wartości $\forall j \in \{1, 2, \dots, n\} : \sum_{i=1}^m p_{ij}$
- 2. Dwa pierwsze zadania ustaw w kolejności zapewniającej krótszy czas wykonania (mniejszą wartość kryterium $C_{\max}$)
- 3. Dla $k := 3$ do n wykonaj
- 4. Wstaw k -te zadanie do częściowej permutacji w to miejsce, dla którego bieżąca wartość kryterium jest najmniejsza (liczba wszystkich miejsc wstawiania wynosi k)

Uzyskana w ten sposób permutacja dla $k = n$, jest wynikiem działania algorytmu NEH

2.6 Algorytmy najtańszego włączania i 2-optimalny dla TSP

2.6.1 Algorytm najtańszego włączania (Cheapest Insertion Algorithm)

Algorytm przybliżony

Złożoność obliczeniowa $O(n^2 \log n)$
Aproksymowalność $\varepsilon = 2$

1. Ustaw $U_1 := \emptyset, U_2 := N$
2. Wybierz miasto $k \in U_2$ jako początek cyklu Hamiltona, zaktualizuj zbiór $U_2 := U_2 - \{k\}$ oraz utwórz ciąg $U_1 := (k)$
3. Znajdź miasto $l \in U_2$ o minimalnym koszcie $c_{l,k} + c_{k,l}$, ustaw $U_2 := U_2 - \{l\}$ oraz zaktualizuj $U_1 := (k \rightarrow l \rightarrow k)$
4. Znajdź łuk $(g \rightarrow h) \in U_1$ i miasto $m \in U_2$, tak aby zminimalizować wartość $c_{g,m} + c_{m,h} - c_{g,h}$
5. Następnie ustaw $U_2 := U_2 - \{m\}$, zaktualizuj cykl $U_1 := (k \rightarrow \dots \rightarrow g \rightarrow m \rightarrow h \rightarrow \dots \rightarrow k)$
6. Sprawdź czy $U_2 = \emptyset$. Jeżeli tak to zakończ algorytm z rozwiązaniem zawartym w U_1 , jeżeli nie to przejdź do kroku 4.

To znaczy $Q^{\text{INS}} \leq \varepsilon Q^*$

2.6.2 Algorytm 2-optimalny

Algorytmy r -optimalne: Algorytm startuje z rozwiązania dopuszczalnego, uzyskanego przez inny algorytm, np. algorytm włączania. Następnie dokonuje się wymiany łuków (krawędzi) w cyklu Hamiltona.

Algorytm przybliżony

Złożoność obliczeniowa $O(n^2 \log n)$
Aproksymowalność $\varepsilon = 2$

1. Dwa niesąsiadujące krawędzie są usuwane i zastępowane przez inne krawędzie tak, aby zachować cykl Hamiltona o mniejszej wartości kryterium
2. Są dokonywane wszystkie możliwe wymiany. Jest ich $\frac{(n-3)n}{2}$

2.7 Algorytm zachłanny dla 0-1 KP

Algorytm przybliżony

Złożoność obliczeniowa $O(n \log n)$
Aproksymowalność $\varepsilon = 2$

1. Sortuj elementy malejąco pod względem wartości $\frac{w_i}{v_i}$
2. Wkładaj elementy do plecaka w kolejności ustalonej w kroku 1, aż do wypełnienia plecaka.

2.8 Algorytm konstrukcyjny dla UFLP

Jest to algorytm zachłanny

Zdefiniujmy zbiory pomocnicze:

- $L' = \{\forall i \in L : x_i = 1\}$ — zbiór uruchomionych lokalizacji
- $C' = \{\forall i \in \{1, 2, \dots, n\}, j \in C : z_{ij} = 0\}$ — zbiór klientów nieprzypisanych do żadnej uruchomionej lokalizacji

Algorytm przybliżony

Złożoność obliczeniowa ?
Aproksymowalność ?

1. Ustaw $L' := \emptyset, C' := C$
2. Wybierz $i \in L - L'$ oraz podzbiór $C'' \subseteq C'$, tak aby

$$(i, C'') = \arg \min_{\substack{i \in L - L' \\ C'' \subseteq C'}} \left(\frac{f_i + \sum_{j \in C''} c_{ij}}{|C''|} \right)$$

3. Uaktualnij $L' := L' \cup \{i\}, C' := C' - C''$
4. Powtarzaj kroki 2. i 3. aż $C' = \emptyset$